23CSE455 Design Patterns

Case Study Report

Team: A5

Case study: Healthcare Application using Laravel: "Design Pattern Identification in a Laravel-Based Healthcare Management Application"

Application: `esteham/hospital-management`. A public Laravel 12 application for patients, appointments, doctors, diagnostic tests, local billing fields and prescriptions.

Date: 2026-10-10

Submission repository: 23CSE455_DP_CaseStudy (team folder `A5/`)

Students

Roll number	Student name
AM.AL.U4AID23014	Nemali Ashrith Reddy
AM.AL.U4AID23062	Akshay Reddy Velugati
AM.SC.U4CSE23212	B. Pravallika
AM.SC.U4CSE23022	Daripineni Manogna

1 Application and architecture

Original source: https://github.com/esteham/hospital-management. Baseline commit `1e7420e431771e3f5eec470d8a005e8255fa5fe8`, dated 17 November 2025. The controllers and models examined for this report are copied in `data/code/`, together with `composer.json`. That folder is not a full clone. The rest of the application is the GitHub commit above. `vendor/` and `.env` are not included.

Environment: PHP ^8.2 and Laravel 12 (`laravel/framework` ^12.0), read from `data/code/composer.json`. The frontend is Vue 3 with Inertia. The database is MySQL through Eloquent. Other confirmed packages are `barryvdh/laravel-dompdf` and `simplesoftwareio/simple-qrcode`. Authentication uses Laravel Sanctum and Breeze. Part 3 was run with Java 17, CPython 3.11.9, Node.js 22.14.0 and g++ (C++17) on Windows.

Scope: We inspected the controllers and models for the six required modules: `AppointmentController`, `Diagnostic/DiagnosticBookingController`, `Doctor/AppointmentController`, and the models `User`, `Appointment`, `Doctor`, `DiagnosticService`, `BookingDiagnostic`, `Prescription`, `PackageBooking` and `TestResult`. There is no `app/Services`, `app/Repositories`, `app/Adapters` or `app/Observers` directory in this commit.

Architecture: Figure 1 is the request path that exists today. A Vue and Inertia page reaches the Laravel router. The router calls a controller. The controller validates the request, applies the business rule, calls a framework facade when it needs mail or a PDF, and writes a row through an Eloquent model. The same path covers the six modules. A patient is a user, a doctor is a linked profile, an appointment or a diagnostic booking is created in its controller, and a prescription is the medical record stored against that appointment.

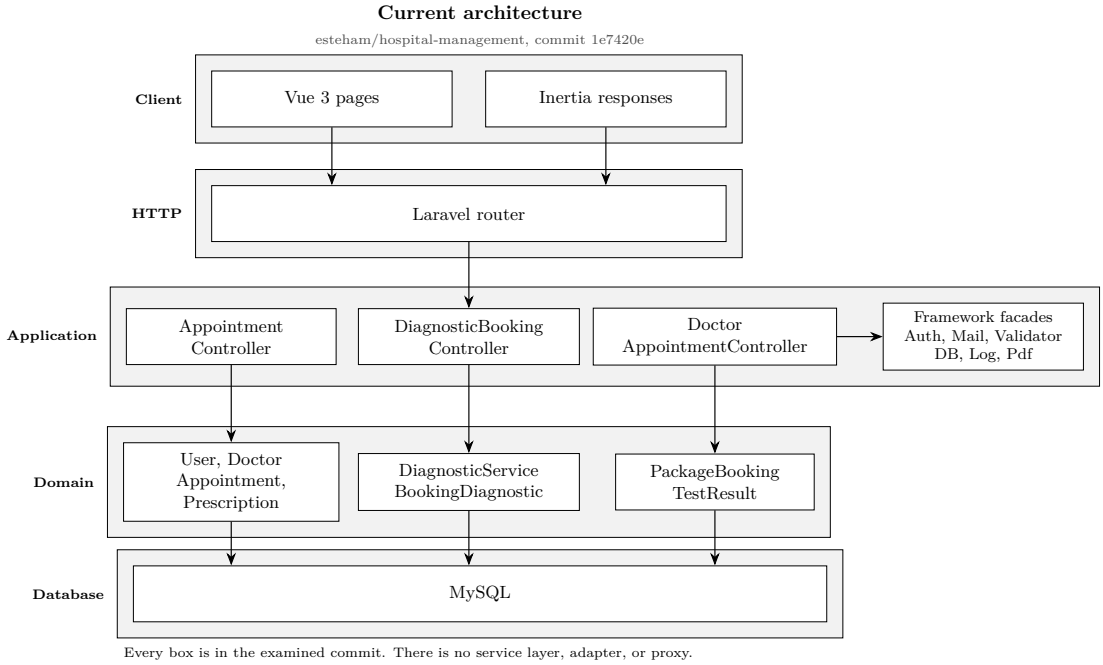


Figure 1. Current architecture. The shaded bands are the layers that exist today.

Run instructions: The original application is run from a clone of the commit above, not from this folder: `composer install`, then `php artisan serve`. The Part 3 programs are the programs we executed. Each language folder under `Part3/` has a README with the command. The printed results are in `data/test_result/`.

2 Part 1 Pattern discovery

What the source actually uses
Active Record: an Eloquent model is the table. `Appointment::create` writes the row.
Facade (framework only): controllers call Laravel facades (`Auth`, `Mail`, `Validator`, `DB`, `Pdf`). There is no application facade class.
Not found: Adapter, Proxy, Strategy, Repository, Observer, and a service layer.

Method: We opened the files in `data/code/`. A pattern was recorded only when a class relationship in those files supports it. A Laravel built-in was labelled as framework code. A name that appears only in a README was not counted. The two types that met that test are Active Record, because the Eloquent model is the table row, and Facade, because the controllers call Laravel's `Mail`, `Pdf`, `Auth`, `Validator`, and `DB`. The payment list and the mail call inside `storePrescription` were read and were not counted as Strategy or Observer.

2.1 Totals at the baseline

Confirmed pattern types: 2: Active Record and Facade.

Facade scope: Framework only. No custom facade was found under `app/`.

Not identified: Adapter, Proxy, Repository, Service layer, Strategy and Observer.

Observation scope: The files listed in Section 1, at commit `1e7420e`.

Rejected candidate	Reason
Adapter	No class translates an external payload into the application's fields.
Proxy	No surrogate stands in front of a model or a remote call.
Strategy	`payment\_method` is the string list `cash`, `card`, `online`. It is not a family of classes.
Repository	Queries are written in the controllers.
Service layer	There is no `app/Services` directory. `DiagnosticService` is an Eloquent model.
Observer	`Mail::to` is called inside `storePrescription`, not from a model observer.
Application Facade	The `use Facade` lines are Laravel's own facades.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Where
Active Record	Appointment row, Diagnostic payment flags, Patient and doctor identity	3	Application
Facade	Prescription mail and PDF, Controller facades	2	Framework only

Data files: `Part1/patterns.csv` and `Part1/Pattern_count.csv`.

2.3 One record for each pattern instance

Record 1: Appointment row

Instance name: Appointment row

Pattern type and category: Active Record. The model is the persistence object.

Purpose: `AppointmentController::store` checks the doctor's daily capacity and then inserts the booking.

Participant mapping

Pattern role	Actual class and function	File and line range
Model	`Appointment`	`data/code/app/Models/Appointment.php:8-39`
Create	`Appointment::create` inside `store`	`data/code/app/Http/Controllers/AppointmentController.php:56-121`
Relation	`Appointment::doctor`, `Appointment::prescriptions`	`data/code/app/Models/Appointment.php:26-39`

Evidence: The capacity check and the insert are in the same controller method. `Schedule` is queried with `whereJsonContains` before the row is created. Mail is sent from that method through the `Mail` facade (lines 111 and 121).

Record 2: Diagnostic payment flags

Instance name: Diagnostic payment flags

Pattern type and category: Active Record, with a framework Facade for validation and PDF.

Purpose: A diagnostic booking is stored locally. No payment gateway is called.

Participant mapping

Pattern role	Actual class and function	File and line range
Model	`BookingDiagnostic`	`data/code/app/Models/BookingDiagnostic.php:7-59`
Validation	`payment\_method` in cash, card, online	`DiagnosticBookingController.php:55`
Facade	`Validator`, `Auth`, `Pdf`	lines 10–14 and 134 of the same controller

Evidence: `store` sets `payment\_status` to `pending` and calls `BookingDiagnostic::create`. `composer.json` does not require Stripe, PayPal or any other payment package. Package bookings are a different table, with `payment\_type`, `amount\_paid` and `total\_amount` (`data/code/app/Models/PackageBooking.php`).

Record 3: Prescription mail and PDF

Instance name: Prescription mail and PDF

Pattern type and category: Framework Facade, used from a fat controller.

Purpose: The doctor who owns the appointment writes `prescription\_text` and emails a PDF.

Participant mapping

Pattern role	Actual class and function	File and line range
Model	`Prescription`	`data/code/app/Models/Prescription.php:7-18`
Controller	`storePrescription`	`data/code/app/Http/Controllers/Doctor/AppointmentController.php:71-94`
Facades	`Pdf::loadView`, `Mail::to`	lines 90 and 94

Evidence: The ownership check, the insert, the PDF and the mail send are one method. The mail send is inside a try/catch in that method. This is not an Observer.

Record 4: Patient and doctor identity

Instance name: Patient and doctor identity

Pattern type and category: Active Record.

Purpose: A patient is a `User` with a role. A doctor profile is a `Doctor` row that belongs to that user. There is no `Patient` model.

Participant mapping

Pattern role	Actual class and function	File and line range
User	`User::doctor`, `User::staff`	`data/code/app/Models/User.php:15-32`
Doctor	`Doctor::user`, `Doctor::schedules`	`data/code/app/Models/Doctor.php:8-16`

Evidence: Appointment rows also store `first\_name`, `last\_name`, `email` and `phone` on the booking itself, so a booking does not require a user foreign key.

3 Part 2 Evolution study

Part 2 does not change commit `1e7420e`. It records what the design would have to become for three external systems. Epic, Quest Diagnostics, Stripe and PayPal are examples. They are not integrations in the repository. Dashed boxes in Figure 2 are proposed. The laboratory and the health record each need a class that translates an outside payload, so both use Adapter. Billing

needs two providers behind one call, so that change uses Strategy. In that design the controller still validates the HTTP request, and the service makes the outside call.

Integration	As coded	Proposed
Laboratory	`DiagnosticBookingController` creates `BookingDiagnostic`	`LaboratoryIntegrationService`, `LaboratoryIntegrationInterface`, `QuestDiagnosticsAdapter`
Billing	Controller stores `payment\_method` and `payment\_status`	`BillingService`, `PaymentStrategy`, `StripePayment`, `PayPalPayment`
Patient data	Controller reads `User`, `Appointment`, `Prescription`	`PatientDataService`, `EHRProvider`, `EpicEHRAdapter`, `EpicFHIRApi`

Pattern	Job	Phase
Adapter	Translates a nested FHIR sample into first name, last name and phone. A laboratory adapter does the same job for a vendor payload.	Part 3 implements the EHR adapter. The Quest adapter stays a proposal.
Strategy	Swaps Stripe and PayPal behind one payment call. The context has no provider branch.	Part 3
Proxy	A cache proxy would return a stored EHR copy when it has one.	Proposal only
Service layer	Moves the orchestration out of the controller.	Described with the two Part 3 patterns
Facade	The existing Laravel facades stay as they are.	Unchanged

Naming rule: The proposed laboratory service is `LaboratoryIntegrationService`. The name `DiagnosticService` is already the catalogue model (`data/code/app/Models/DiagnosticService.php:7`).

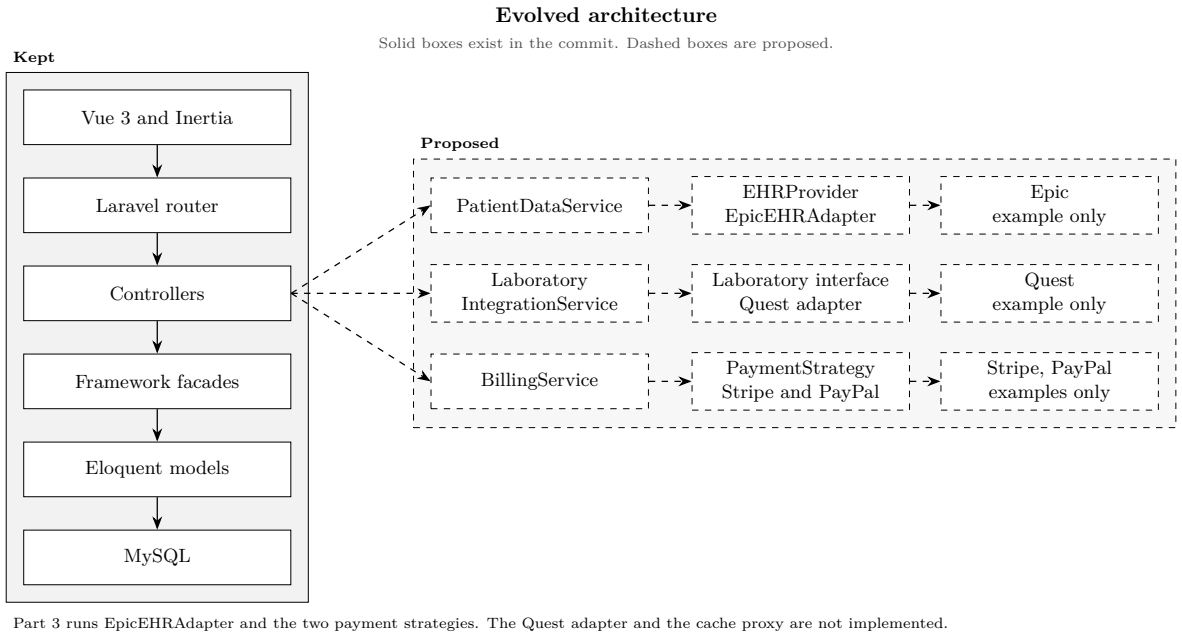


Figure 2. Evolved architecture. The left band is the application we inspected. The dashed band is proposed.

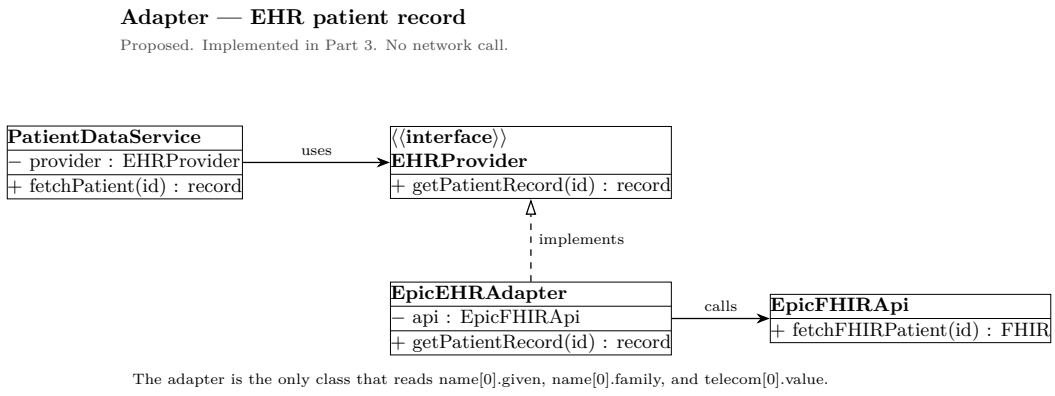


Figure 3. Adapter class diagram for the proposed EHR integration.

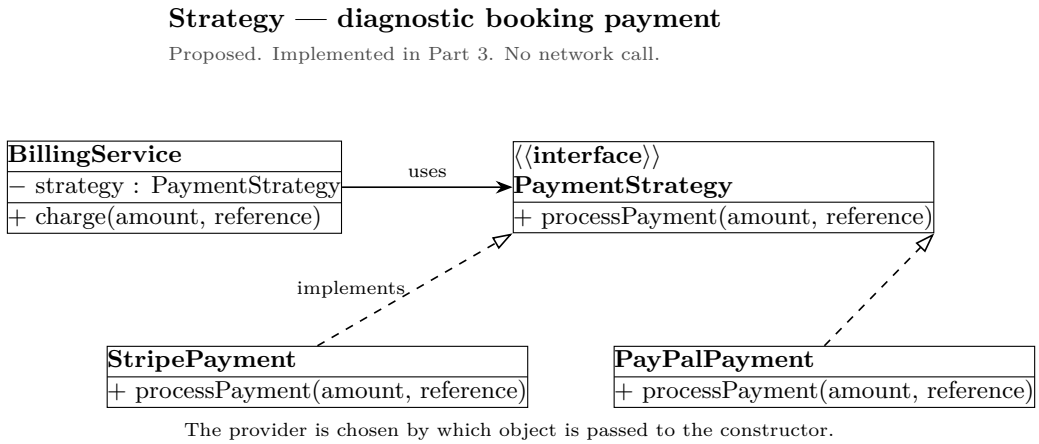


Figure 4. Strategy class diagram for the proposed billing providers.

Refactoring: In the original design the business rule stays in the controller. After the proposed refactoring the controller still validates HTTP input. It does not parse FHIR and it does not switch on a provider name. This refactoring has not been applied to the upstream repository. The notes for the three changes are in `Part2/changes/Change1/`, `Change2/` and `Change3/`. The proposed PHP roles are in `Part2/code/`.

4 Part 3 Cross-language implementations

The two patterns written in Part 3 are Adapter and Strategy. The four languages are Java, Python, JavaScript and C++. The original application remains Laravel and PHP. None of the programs opens a network connection. Each program builds the concrete adapter or strategy at the start and passes it into the client through the constructor. All four languages use patient `PT-98765` and the references `DX-1001` and `DX-1002`, and the files in `data/test_result/` are the output of those runs.

Language	Role	Class
Java, Python, C++	Adapter target	`EHRProvider`
JavaScript	Adapter target	the same method names, with no compile-time interface
All four	Adapter client	`PatientDataService`
All four	Strategy context	`BillingService`
All four	Concrete strategies	`StripePayment` and `PayPalPayment`

Language	Version used	Test command
Java	Java 17	`javac AdapterDemo.java` then `java AdapterDemo`
Python	CPython 3.11.9	`python adapter_demo.py`
JavaScript	Node.js 22.14.0	`node adapterDemo.js`
C++	g++, C++17	`g++ -std=c++17 adapter_demo.cpp -o adapter_demo` then run it

The Strategy commands are the same, with the file names in `Part3/strategy/`.

Actual results: The adapter programs print `first\_name` John, `last\_name` Smith and phone `555-1234` for patient `PT-98765`. The strategy programs print provider `stripe` for reference `DX-1001` and provider `paypal` for reference `DX-1002`. The captured output is in `data/test_result/`. The JavaScript and C++ adapter programs exit with code 1 if those three patient fields are wrong. The JavaScript and C++ strategy programs exit with code 1 if the provider name is wrong.

Comparison:

- The client receives the interface through its constructor in every language. The concrete adapter or strategy is built at the start of the program.
- Java and C++ state the contract as an interface or an abstract class. Python uses an abstract base class. JavaScript uses the same method names without a compile-time interface.
- Python spells the adapter method `get\_patient\_record`. The other three languages spell it `getPatientRecord`.
- The comparison files are `Part3/adapter/pattern_crosslanguage.csv` and `Part3/strategy/pattern_crosslanguage.csv`. The source-location column is the non-blank line count of each program. Cyclomatic complexity and process time were not measured, so those two columns say "not measured".

5 LLM evidence

An LLM drafted explanations and the first versions of the four-language programs. A claim about the repository was kept only when the file in `data/code/` supported it. During the investigation the LLM could not read the GitHub tree, so the module map was rebuilt from the files we opened. The working thread is https://share.gemini.google/zjKon9gmTNO8.

Task	What was drafted	What was kept after checking
Choose the application	Three public Laravel repositories	The empty skeleton was dropped. This report uses `esteham/hospital-management`.
Name existing patterns	Several GoF names, including Adapter	Adapter, Proxy, Strategy, Repository, Observer and a service layer are marked not identified.
Evolve the design	Adapter, Strategy, Proxy and services	Billing is Strategy. The proxy and the Quest adapter stay proposals.
Laboratory service name	The first draft reused `DiagnosticService`	Renamed to `LaboratoryIntegrationService`.
Part 3 languages	PHP and TypeScript were drafted first	The submitted languages are Java, Python, JavaScript and C++, as required.

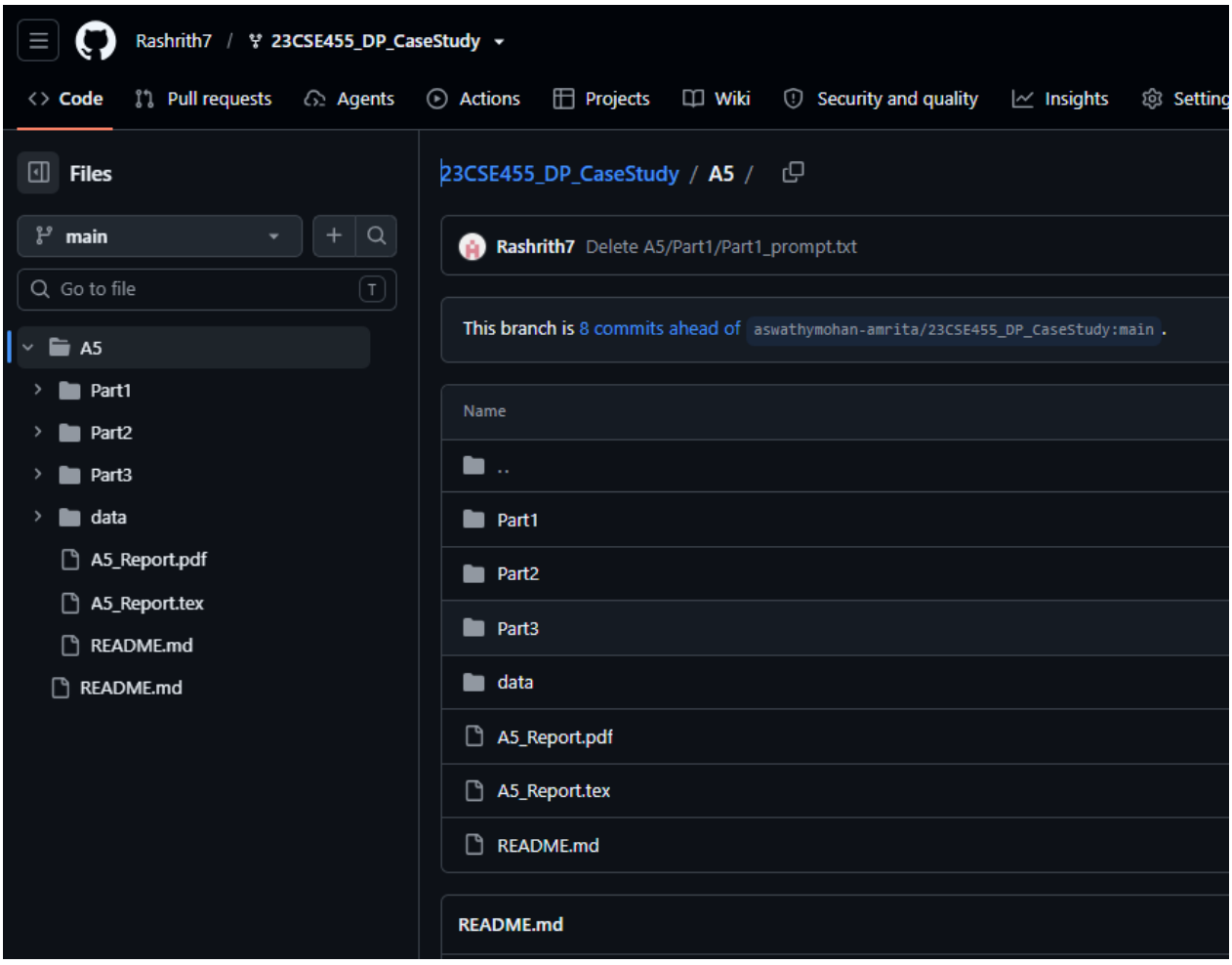
The check against the Gemini thread is `Part1/llm/verification.md`. The record files are `Part1/llm.csv`, `Part2/llm.csv`, `Part3/adapter/llm/llm.csv` and `Part3/strategy/llm/llm.csv`.

6 Contributions

The four members divided the case study into equal parts.

Student	Work
Nemali Ashrith Reddy	Part 1. Inspection of the hospital modules, the pattern inventory, and the current architecture.
Akshay Reddy Velugati	Part 2. The laboratory, billing, and patient-data evolution, with the evolved architecture and the UML diagrams.
B. Pravallika	Part 3, Adapter. The same EHR adapter in Java, Python, JavaScript, and C++, and the captured adapter output.
Daripineni Manogna	Part 3, Strategy. The same payment strategy in Java, Python, JavaScript, and C++, and the captured strategy output.

7 Screenshot



8 Conclusion

`esteham/hospital-management` at commit `1e7420e` implements the six healthcare areas with Eloquent models and with controllers that hold the business rules. The patterns supported by that source are Active Record and Laravel's own facades. Adapter, Strategy, Proxy, Repository, Observer and a service layer were not found.

Adapter is the translation point if an electronic health record is added. Strategy is the switch if a

real payment provider is added. Those two patterns are implemented in Java, Python, JavaScript and C++. The laboratory adapter and the cache proxy remain proposals, so this report does not describe code that was not written.

References

[1] Esteham H. Zihad Ansari. Hospital Management. https://github.com/esteham/hospital-management.

[2] E. Gamma, R. Helm, R. Johnson and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[3] Laravel 12 documentation, facades and Eloquent. https://laravel.com/docs/12.x.

9 Supplement: Eight-pattern extension (11 October 2026)

Evidence boundary. The original Laravel baseline remains two observed pattern types: Eloquent Active Record and Laravel framework Facade. The following eight collaborations are new standalone PHP 8.2+ demonstrations, not discoveries in the baseline, nor integrated Laravel routes or live provider APIs. Each uses synthetic inputs and fake providers. This extension is independent of the A8 NestJS case study.

Proposed team allocation	Two patterns	Healthcare use cases
Nemali Ashrith Reddy	Facade; Repository	Appointment booking; patient persistence
Akshay Reddy Velugati	Adapter; Strategy	Lab order translation; payment selection
B. Pravallika	Proxy; Observer	Patient cache; clinical event notification
Daripineni Manogna	Factory Method; Decorator	Record export; read-access audit

Implementation. The source is `Part2/eight_patterns/HealthcarePatterns.php`; `test.php` contains fourteen contract assertions. The inventory is `Part2/eight_patterns/pattern_inventory.csv`. Run from `A5/`: `php Part2/eight_patterns/test.php`. Results are not claimed until this command is actually executed.

Pattern roles. AppointmentFacade delegates to AppointmentStore and NoticeSender. PatientRegistry depends on PatientRepository. LaboratoryAdapter translates to ExternalLabClient. BillingContext selects PaymentStrategy. CachedPatientProxy implements PatientDataSource and caches its wrapped source. ClinicalEventPublisher dispatches to ClinicalObserver subscribers. ExporterFactory creates RecordExporter implementations via a factory method. AuditedMedicalRecordReader decorates MedicalRecordReader with audit logging.

Evolution impact. Proposed dependencies flow from client to interface to fake or production adapter; substitution supports isolated tests. The new cache requires invalidation, the audit wrapper requires access controls, and billing needs real idempotency, reconciliation and gateway signature verification before production deployment. These risks are not addressed by the fakes alone.

Limitations. This is not a full Laravel integration, and no external vendor has been contacted. Thirty-two standalone programs now cover eight patterns in Java, Python, JavaScript and C++ (four implementations per pattern). They have not been verified by a successful CI run. The earlier Adapter and Strategy demonstrations remain separate. Complexity, runtime benchmarks and revised PDF verification are pending. Team allocations are proposed and must be verified with members; no individual authorship is inferred from this document.

9.1 Cross-language implementation matrix

All eight proposed patterns have one reduced implementation in each of four languages (32 source files). These examples use synthetic healthcare data and deterministic assertions, not production endpoints. The reproducible commands are recorded in `Part3/eight_patterns/cross_language.csv`. The automated validation workflow is `.github/workflows/a5-validate.yml`.

Pattern	Python	JavaScript	Java	C++
Facade	source	source	source	source
Repository	source	source	source	source
Adapter	source	source	source	source
Strategy	source	source	source	source
Proxy	source	source	source	source
Observer	source	source	source	source
Factory Method	source	source	source	source
Decorator	source	source	source	source

9.2 Verification status and measurement protocol

The GitHub workflow is configured to execute the PHP contract tests and 32 cross-language programs, then compile this report twice with `pdflatex`. A configured workflow is not evidence of a successful run. All newly added test results remain *unverified* until CI logs or local execution outputs are saved. No benchmark numbers are fabricated. Future measurements should record compiler/runtime versions, five timed runs after warm-up, source-only nonblank LOC and a defined cyclomatic complexity convention.

9.3 Scope of integration and ethical claims

The repository retains only selected Laravel source excerpts in `data/code/`; it does not include a full runnable Laravel checkout with routes, migrations, framework bootstrap, dependencies and configuration. Consequently the new PHP examples are *not* integrated into an executable Laravel application. They demonstrate design evolution and testable boundaries. Integration with real providers requires authorization, secrets management, consent, patient-data minimization, timeouts, idempotency, webhook verification, database transactions and staging tests. The proposed division of work is not proof of individual authorship.